

ANNEXE 1

Architecture Multi-couche

Sommaire

Image :

1) Couche Présentation

2) Couche Traitement / Métier

3) Couche Physique / Données

4) Récapitulatif

Texte :

5) Pourquoi avoir une architecture multi-couches ?

6) Couche Présentation

7) Couche Traitement / Métier

8) Couche Physique / Données

9) Le point important dans notre cas

1) Schéma Présentation

Couche présentation

Écran, saisie et affichage utilisateur

RECEIVE / RECEIVE MAP

Réception de la saisie

SEND TEXT / SEND MAP

Affichage du résultat

2) Schéma traitement / métier

Couche traitement / métier

Règles de gestion et orchestration

Contrôles de conformité

Doublons, formats, valeurs

PERFORM / EVALUATE

Orchestration des paragraphes

3) Schéma physique / données

Couche physique / données

Accès direct au support de stockage

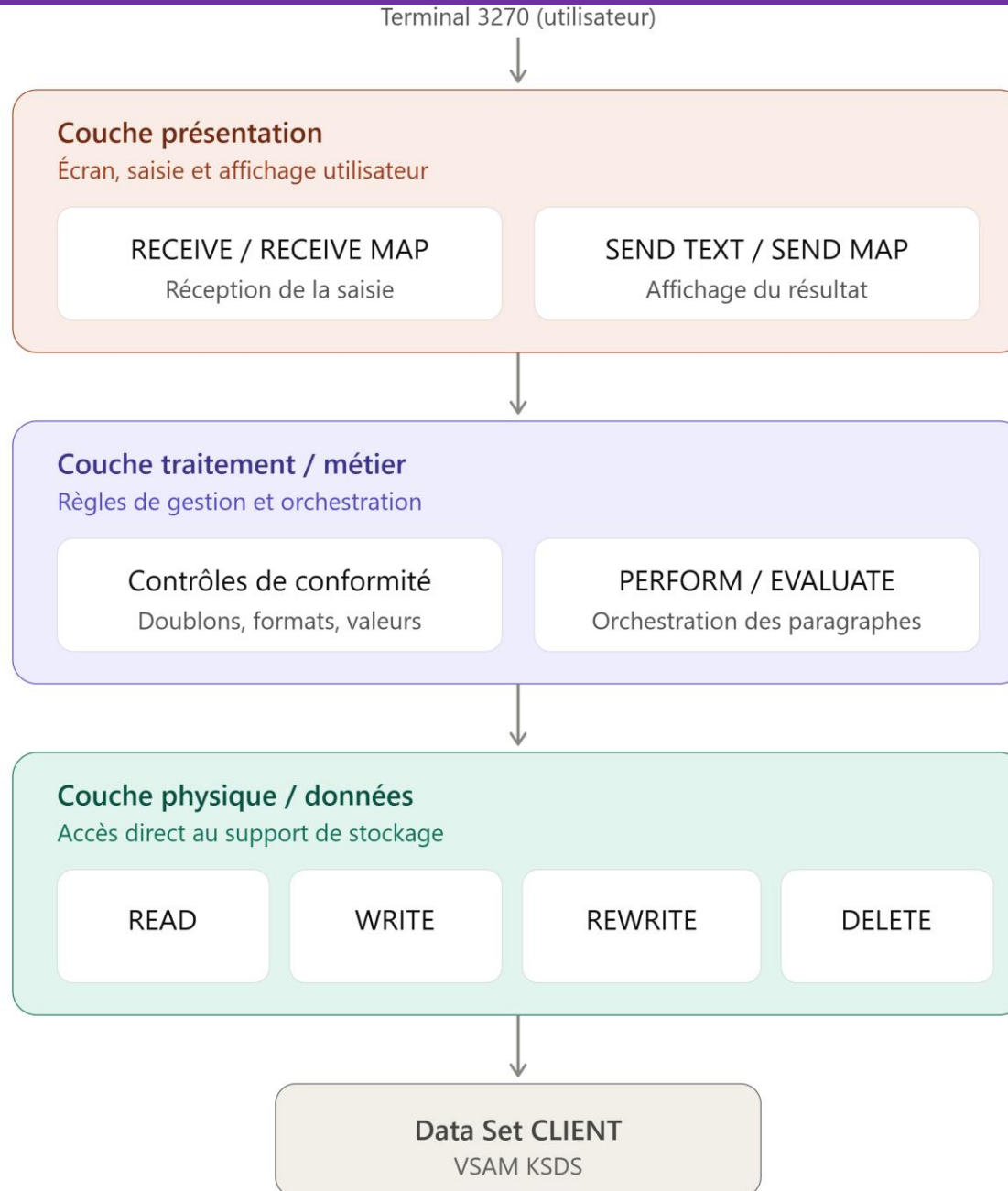
READ

WRITE

REWRITE

DELETE

4) Schéma récapitulatif



5) Pourquoi avoir une architecture multi-couches ?

L'idée centrale est de **séparer les responsabilités** dans un programme pour :

- Isoler ce qui peut changer indépendamment (l'écran, la logique métier, le support de stockage)
- Faciliter la maintenance (modifier une couche sans casser les autres)
- Permettre la réutilisation (le même traitement métier pourrait servir plusieurs interfaces)
- Clarifier les responsabilités (chaque couche a un rôle précis)

En mainframe CICS/COBOL, on retrouve classiquement **3 couches**.
A savoir qu'il est possible d'avoir des « couches intermédiaires »

6) Couche Présentation

- **Rôle** : gérer l'échange avec l'utilisateur/l'écran.
- **Dans un vrai projet** : utilise des **MAP BMS** (Basic Mapping Support) — des écrans formatés avec des zones de saisie définies précisément (position, longueur, attributs), gérées par EXEC CICS SEND MAP / RECEIVE MAP.
- **Dans notre TP actuel (simplifié)** : on utilise SEND TEXT (affichage texte brut) et on a testé RECEIVE non mappé (qu'on a vu être peu fiable)

Ex :
PARA-RECEPTION-DONNEE.
 MOVE '000001' TO WS-NUM-COMPTE-SAISI.
...
PARA-AFFICHAGE-RESULTAT.
 EXEC CICS SEND TEXT ...

7) Couche Traitement / Métier

- **Rôle** : contient la **logique applicative** — règles de gestion, contrôles de cohérence, calculs, orchestration des étapes. C'est le cœur "intelligent" du programme, indépendant de la façon dont les données arrivent (écran, batch, appel d'un autre programme) et de la façon dont elles sont stockées (VSAM, DB2, fichier séquentiel...).
- Exemple dans **PRGREAD** :

PARA-CONTROLE-DONNEE.

IF WS-NUM-COMPTE-SAISI = SPACES

SET TRAITEMENT-KO TO TRUE ...

C'est ici pour PRGWRITE qu'on vérifiera la conformité des champs et l'absence de doublon.

C'est la logique de gestion pure, sans se soucier de comment on n'affiche ni de comment on écrit physiquement dans le VSAM

8) Couche Physique / Données

- **Rôle** : gérer l'accès concret au support de stockage — ici un Data Set VSAM KSDS, via les commandes EXEC CICS READ / WRITE / REWRITE / DELETE.
- Exemple dans **PRGREAD** :

PARA-LECTURE-CLIENT.

EXEC CICS READ

FILE('CLIFI')

INTO(WS-ENREG-CLIENT)

RIDFLD(WS-NUM-COMPTE-SAISI)

RESP(WS-RESP)

END-EXEC

9) Le point important dans notre cas

Contrairement à une architecture multi-couches « physique » (où chaque couche serait un **module/programme séparé**, communiquant via EXEC CICS LINK avec des COMMAREA, comme on ferait en environnement de production réel),

ici les 3 couches sont matérialisées **logiquement** par des paragraphes bien distincts et commentés à l'intérieur d'un seul programme COBOL.

C'est une architecture multi-couches « **didactique** » .

C'est pour apprendre la séparation des responsabilités sans la complexité additionnelle de la communication inter-programmes.